

## All files server.js

**34.02%** Statements 49/144    **39.72%** Branches 29/73    **20%** Functions 5/25    **35.25%** Lines 49/139

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```

1      /* eslint-env node, es2021 */
2      import express from "express";
3      import dotenv from "dotenv";
4      import path from "path";
5      import fs from "fs";
6      import { fileURLToPath } from "url";
7      import { spawn } from "child_process";
8
9      1x  dotenv.config();
10
11      1x  const __filename = fileURLToPath(import.meta.url);
12      1x  const __dirname = path.dirname(__filename);
13
14      1x  export const app = express();
15      1x  const PORT = process.env.PORT || 3000;
16
17      1x  const clientId = process.env.CLIENT_ID;
18      1x  const clientSecret = process.env.CLIENT_SECRET;
19
20      1x  const redirectUri = "http://localhost:3000/auth/callback";
21      1x  const tenant = "common";
22
23      1x  app.use(express.json());
24      1x  app.use(express.static(__dirname));
25
26      // const dataDir = path.join(__dirname, "data");
27      1x  const dataDir = path.join(__dirname, process.env.DATA_DIR || "data");
28
29
30
31      1x  I if (process.env.NODE_ENV !== "test") {
32          app.listen(PORT, () => {
33              console.log(`Server running → http://localhost:${PORT}`);
34          });
35      }
36
37      1x  E if (!fs.existsSync(dataDir)) {
38      1x  fs.mkdirSync(dataDir);
39      }
40      //-----
41      //TODO dont let professors change their email or ID to someone else / verify that its their info
42      // -shove their email into facultyId and disable switching
43      // -verify that it is a @dsu.edu email path
44      //TODO Professors should only have to rate courses they would teach anyway, not random things
45      // -dropdown multiple checklist so profs in each college only has to rate classes in their college
46      // -ie Beacom - MATH,CSC,CSI etc
47      // -use class info for filter, then only show through that filter
48      //-----
49      1x  const solutionFilePath = path.join(dataDir, "solution.xlsx");
50
51      1x  app.get("/login", (req, res) => {
52          const url =
53      1x  `https://login.microsoftonline.com/${tenant}/oauth2/v2.0/authorize` +
54      `?client_id=${clientId}` +

```

```
55   `&response_type=code` +
56   `&redirect_uri=${encodeURIComponent(redirectUri)}` +
57   `&response_mode=query` +
58   `&scope=openid profile email`;
59
60 1x   res.redirect(url);
61   });
62   //connect to faculty.html
63 1x   app.get("/faculty", (req, res) => {
64 1x     res.sendFile(path.join(__dirname, "faculty.html"));
65   });
66
67   function runSolverScript() {
68     return new Promise((resolve, reject) => {
69       const solverPath = path.join(__dirname, "solver.py");
70
71       if (!fs.existsSync(solverPath)) {
72         return reject(
73           new Error(
74             `Solver script not found at ${solverPath}. Ask Ryan or IT.`
75           )
76         );
77       }
78
79       //change "python3" to "python" or vice versa if needed for environment
80       const pythonCMD = process.env.PYTHON || "python";
81
82       const proc = spawn(pythonCMD, [solverPath], {
83         cwd: __dirname,
84       });
85
86       let stdout = "";
87       let stderr = "";
88
89       proc.stdout.on("data", (chunk) => {
90         stdout.out += chunk.toString();
91       });
92
93       proc.stderr.on("data", (chunk) => {
94         stderr += chunk.toString();
95       });
96
97       proc.on("error", (err) => {
98         reject(err);
99       });
100
101       proc.on("close", (code) => {
102         if (code === 0) {
103           resolve({ stdout, stderr, code });
104         } else {
105           const err = new Error(
106             `Solver exited with code ${code}${
107               stderr ? `: ${stderr}` : ""
108             }`
109           );
110           err.code = code;
111           err.stdout = stdout;
112           err.stderr = stderr;
113           reject(err);
114         }
115       });
116     });
```

```
117     }
118     //if preference for term exists, add to it
119     function appendSubmittedPreference(pref) {
120         const {
121             facultyId,
122             termCode,
123             termLabel,
124             classId,
125             classLabel,
126             rating,
127             desireRating,
128 1x     } = pref;
129 1x     if (!facultyId || !termCode || !classId) {
130         throw new Error("Missing facultyId, termCode, or classId");
131     }
132
133     //Sanitize data for filesystem (specifically any crazy user names)
134 1x     const safeFaculty = String(facultyId)
135         .trim()
136         .toLowerCase()
137         .replace(/[^\a-z0-9@._-]/gi, "_")
138         .replace(/[@.]/g, "_");
139
140 1x     const fileName = `pref_${safeFaculty}_${termCode}.csv`; //connect and name appropriately the file
141 1x     const filePath = path.join(dataDir, fileName); // shove to proper folder
142
143     const header =
144 1x         "timestamp,facultyId,termCode,termLabel,classId,classLabel,rating,desireRating";
145
146 1x     const scrub = (v) =>
147 6x         String(v ?? "")
148             .replace(/,/g, " ")
149             .replace(/\r?\n/g, " ");
150
151 1x     const timestamp = new Date()
152         .toISOString()
153         .replace("T", " ")
154         .substring(0, 19);
155
156 1x     const newRow = [
157         scrub(timestamp),
158         scrub(facultyId),
159         scrub(termCode),
160         scrub(termLabel),
161         scrub(classId),
162         scrub(classLabel),
163         Number.isFinite(Number(rating)) ? Number(rating) : "",
164         Number.isFinite(Number(desireRating)) ? Number(desireRating) : "",
165     ].join(",");
166
167 1x     if (!fs.existsSync(filePath)) {
168 1x         const content = header + "\n" + newRow + "\n";
169 1x         fs.writeFileSync(filePath, content, "utf8"); //header labels
170 1x         return;
171     }
172
173     //file exists -> read and update/replace
174     const raw = fs.readFileSync(filePath, "utf8");
175     const lines = raw.split(/\r?\n/).filter((l) => l.trim().length > 0);
176
177     const dataLines = [];
178     for (const line of lines) {
```

```
179     if (line.trim() === header) continue;
180     const cols = line.split(",");
181     if (cols.length < 8) continue;
182     dataLines.push(line);
183 }
184
185 const filtered = [];
186
187 for (const line of dataLines) {
188     const cols = line.split(",");
189     const existingTermCode = cols[2] ?? "";
190     const existingClassId = cols[4] ?? "";
191
192     // if same termCode + classId, drop old row
193     if (
194         existingTermCode === String(termCode) &&
195         existingClassId === String(classId)
196     ) {
197         continue;
198     }
199     filtered.push(line);
200 }
201
202 filtered.push(newRow);
203
204 //Rewrite entire file: header + all rows
205 const output = header + "\n" + filtered.join("\n") + "\n";
206 fs.writeFileSync(filePath, output, "utf8");
207 }
208
209 1x app.post("/api/preferences/submit", (req, res) => {
210 2x     const body = req.body || {};
211     const {
212         facultyId,
213         termCode,
214         termLabel,
215         classId,
216         classLabel,
217         rating,
218         desireRating,
219 2x     } = body;
220
221 2x     if (
222         !facultyId ||
223         !termCode ||
224         !classId ||
225         rating === undefined ||
226         rating === null ||
227         desireRating === undefined ||
228         desireRating === null
229     ) {
230 1x         return res.status(400).json({
231             message:
232                 "Missing required fields (facultyId, term, class, rating).",
233         });
234     }
235
236 1x     const rNum = Number(rating);
237 1x     if (!Number.isInteger(rNum) || rNum < 0 || rNum > 4) {
238         return res.status(400).json({
239             message: "Rating must be an integer between 0 and 4",
240         });
241     }
```

```
241     }
242
243 1x   try {
244 1x     appendSubmittedPreference({
245         facultyId,
246         termCode,
247         termLabel: termLabel || "",
248         classId,
249         classLabel: classLabel || "",
250         rating: rNum,
251         desireRating,
252     });
253
254 1x     return res.json({
255         message: "Preference saved and submitted successfully.",
256     });
257   } catch (err) {
258     console.error("Error submitting preference:", err);
259     return res.status(500).json({
260         message: "Server error while submitting preference.",
261     });
262   }
263   });
264   // prettier-ignore
265 1x app.post("/api/admin/solve", async(req, res) => {
266   try {
267     const result = await runSolverScript();
268
269     //make sure the file actually exists
270     const solutionExists = fs.existsSync(solutionFilePath);
271
272     return res.json({
273         success: true,
274         message: solutionExists
275             ? "Solver completed and solution.xlsx was generated."
276             : "Solver completed, but solution.xlsx was not found. Check solver script output.",
277         stdout: result.stdout,
278         stderr: result.stderr,
279     });
280   } catch (err) {
281     console.error("Error running solver:", err);
282     return res.status(500).json({
283         success: false,
284         message: err.message || "Error running solver script.",
285     });
286   }
287   });
288   // prettier-ignore
289 1x app.get("/auth/callback", async(req, res) => {
290     const code = req.query.code;
291
292     const tokenResponse = await fetch(
293         `https://login.microsoftonline.com/${tenant}/oauth2/v2.0/token`,
294         {
295             method: "POST",
296             headers: { "Content-Type": "application/x-www-form-urlencoded" },
297             body: new URLSearchParams({
298                 client_id: clientId,
299                 client_secret: clientSecret,
300                 grant_type: "authorization_code",
301                 code,
302                 redirect_uri: redirectUri,
```

```
303     })),
304   }
305   );
306
307   const tokenData = await tokenResponse.json();
308   res.send(tokenData);
309 });
310
311 1x app.get("/getNames", (req, res) => {
312   fs.readdir(dataDir, (err, files) => {
313     if (err) {
314       return res.status(500).json({ error: "Could not read data directory" });
315     }
316
317     // Only .csv files
318     const csvFiles = files.filter(f => f.endsWith(".csv"));
319
320     const names = csvFiles.map(filename => {
321       const raw = filename.replace(".csv", "");
322
323       // Split by underscores
324       const parts = raw.split("_");
325       // Expected pattern:
326       // pref_<first>_<last...>_dsu_edu_<date>
327
328       // Find "dsu" which marks the end of the name section
329       const dsuIndex = parts.indexOf("dsu");
330
331       // Name is everything from index 1 up to "dsu"
332       // Example: ["pref", "phoenix", "campbell", "dsu", "edu", "202510"]
333       let nameParts;
334
335       if (dsuIndex > 1) {
336         nameParts = parts.slice(1, dsuIndex); // Could be ["mary", "ann", "smith"]
337       } else {
338         // Fallback: just use everything except pref and date
339         nameParts = parts.slice(1, parts.length - 1);
340       }
341
342       // Capitalize each part
343       const formattedName = nameParts
344         .map(part => part.charAt(0).toUpperCase() + part.slice(1).toLowerCase())
345         .join(" ");
346
347       return formattedName;
348     });
349
350     // Remove duplicates and sort alphabetically
351     const uniqueSortedNames = [...new Set(names)].sort((a, b) =>
352       a.localeCompare(b)
353     );
354
355     res.json(uniqueSortedNames);
356   });
357 });
358
359 // POST endpoint that calls Python
360 1x app.post("/downloadFile", (req, res) => {
361   const { name } = req.body;
362   if (!name) return res.status(400).send("Name not provided");
363
364   // Call Python script with the name as an argument
```

```
365     const py = spawn("python3", ["preferences.py", name]);
366
367     let output = "";
368     let errorOutput = "";
369
370     py.stdout.on("data", (data) => {
371         output += data.toString();
372     });
373
374     py.stderr.on("data", (data) => {
375         errorOutput += data.toString();
376     });
377
378     py.on("close", (code) => {
379         if (code !== 0) {
380             console.error("Python error:", errorOutput);
381             return res.status(500).send("Python script error");
382         }
383
384         // Use Python output as file content
385         res.setHeader("Content-Disposition", `attachment; filename="${name}_file.txt"`);
386         res.setHeader("Content-Type", "text/plain");
387         res.send(output);
388     });
389 }
390
391
```

Code coverage generated by [istanbul](#) at 2025-12-01T03:57:45.343Z